

Parallel Compare-and-Swap

1 Problem

We will evaluate against a chain replication protocol that uses an atomic register to implement reconfiguration. States to this register contain a version number; it can only be committed if the version number is exactly the previous one + 1.

The classic distributed shared register algorithms (ABD [1] and its variants) assume that the read value doesn't have to be committed, since they don't implement compare-and-swap, so they allow reading an uncommitted value, attempting to recommit it, and returning that value if the recommit fails. Distributed CAS [2] assumes sequential processing of messages. Hydro encourages batch processing, which requires careful thinking about what can be parallelized.

Our goal is to allow reads to proceed in parallel and minimize changes to the protocol otherwise; our changes should be obviously correct, and more heavy-weight changes should be left to the optimizer.

2 Algorithm Sketch

1. Reads proceed in parallel without a ballot, as in ABD. They are broadcasted to all replicas, wait for a majority, and if everyone agrees on the value, returned to the client.
2. If replicas disagree, the highest-ballot value must be reconciled (by committing the previous write) before the read returns whatever the majority ultimately agrees on. This might not be the reconciled value, if this node loses election.
3. Writes proceed sequentially. During election, a new ballot is chosen, broadcast to all replicas, and the previous value read from the majority.
4. If the majority disagree, the ballot is used to commit the highest-ballot value.
5. Once the majority respond and agree, the write is committed if the CAS condition is met, and rejected otherwise.

Question: When multiple parallel reads fail, what value do we use to reconcile? Can the reconciled values break the CAS condition, based on the order we reconcile?

Proposal: Each failed read knows the highest-ballot value. It should attempt to reconcile with that value *if no other value with a higher ballot has been committed before*. If a value with a higher ballot has been committed, the commit must have happened after the read, so the read can just return that value.

References

- [1] Hagit Attiya, Amotz Bar-Noy, and Danny Dolev. Sharing memory robustly in message-passing systems. *J. ACM*, 42(1):124–142, January 1995. doi: 10.1145/200836.200869.
- [2] Denis Rystsov. Caspaxos: Replicated state machines without logs. *arXiv preprint arXiv:1802.07000*, 2018.